E-Commerce System Architecture Document: Seasonal Fruit Farm Storefront

Project Overview: An end-to-end e-commerce application designed for a small seasonal farmer specializing in mangoes and seasonal fruits. The system supports out-of-state web sales, features dynamic packaging options (By the Pound, Small Box, Large Box), incorporates a manual farmer approval workflow before processing payments, and logs comprehensive visitor traffic metrics (IP, City, State, Browser) for marketing and fraud prevention.

---

1. System Architecture Overview

The application leverages a modern, low-cost, serverless stack optimized for rapid development with AI coding assistants (Cursor / Claude). It ensures a professional frontend look, a robust relational backend, and minimal overhead during off-season periods.

Layer	Technology Selected	Purpose & Hosting Location
Frontend	Next.js (React Framework) + Tailwind CSS	Hosted on **Vercel (Free Tier)**. Provides professional UI, built-in image optimization for fruits, and fast out-of-state SEO.
Backend / API	Next.js Server Actions & Middleware	Hosted on **Vercel (Free Tier)**. Executes secure database operations and handles geographic visitor tracking.
Database	PostgreSQL (via Supabase)	Hosted on **Supabase (Free Tier)**. Relational data storage for tracking orders, analytics, and marketing opt-ins.
Payments	Stripe API (Auth & Capture Mode)	External Gateway. Holds customer funds securely without finalizing charges until farmer approval.

Layer	Technology Selected	Purpose & Hosting Location
Notifications	Resend API	External Mailer. Triggers transactional receipts and administrative alerts.

---

2. Core Workflows & Logic

A. Customer Order & Tracking Workflow

1. **Traffic Capture:** Next.js middleware intercepting incoming requests extracts the IP Address, User-Agent (Browser/Device), and calls a lightweight GeoIP service to derive the City and State. This is instantly recorded to the audit logs.
2. **Checkout:** The user configures their cart selecting by the Pound, Small Box (8 lbs), or Large Box (18 lbs). At checkout, they enter their out-of-state shipping details.
3. **Stripe Pre-Authorization:** Payment credentials are verified via Stripe Checkout utilizing Capture: Manual. Funds are placed on hold for up to 7 days, ensuring no merchant processing fees are forfeited if the harvest or shipping constraints force a cancellation.
4. **Database Entry:** The order is logged in Supabase with a default state of pending_approval.

B. Admin Dashboard & Approval Workflow

1. **Secure Authentication:** The farmer logs into a protected route (/admin) using email-based passwordless Magic Links managed via NextAuth or Supabase Auth.
2. **Order Review:** The admin dashboard isolates all pending_approval entries, matching order destination states with the internal traffic log to flag potential geographic anomalies (fraud vectors).
3. **Action Triggers:**
    - **Approve:** Communicates with Stripe to execute a payment capture, signals the Resend API to send a shipment confirmation email, and updates the database to approved.
    - **Cancel:** Releases the Stripe hold completely, notifies the client via email, and updates the status to cancelled.

---

3. Database Schema Blueprint

Execute this foundational schema directly inside the Supabase SQL editor to scaffold the relation networks:

```
-- 1. Visitor Security & Analytics Log
CREATE TABLE site_traffic (
  id BIGSERIAL PRIMARY KEY,
```

```
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text,
now()) NOT NULL,
  ip_address TEXT,
  city TEXT,
  state TEXT,
  browser TEXT,
  device_type TEXT,
  page_visited TEXT
);

-- 2. Customer & Marketing Table
CREATE TABLE customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text,
now()) NOT NULL,
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  opt_in_marketing BOOLEAN DEFAULT FALSE
);

-- 3. Core Orders Table
CREATE TABLE fruit_orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT timezone('utc'::text,
now()) NOT NULL,
  customer_id UUID REFERENCES customers(id),
  shipping_address JSONB NOT NULL, -- Format: { street: "", city: "",
state: "", zip: "" }
  order_type TEXT NOT NULL,         -- 'pound', 'small_box',
'large_box'
  weight_description TEXT,         -- e.g., '15 lbs', '8 lbs (Small
Box)'
  total_price DECIMAL(10,2) NOT NULL,
  status TEXT DEFAULT 'pending_approval', -- pending_approval,
approved, shipped, cancelled
  stripe_intent_id TEXT,
  traffic_log_id BIGINT REFERENCES site_traffic(id)
);
```

---

4. Prompts Guide for Cursor / Claude Development

Utilize these exact prompts sequentially in Cursor (via `Cmd+I` / Composer or `Cmd+L` / Chat) to build the core architecture chunks:

Phase 1: Project Setup & Product Pricing Page

"Act as a Senior Frontend Engineer. Initialize a Next.js project using Tailwind CSS and the App Router. Create a professional, clean UI component representing an organic fruit farm product page for premium seasonal mangoes. Design a toggle selector allowing the user to select buying options: By the Pound ($6.50/lb), Small Box (8 lbs - $45.00), or Large Box (18 lbs - $85.00). Ensure the theme uses a clean agricultural design with soft warm and earthy slate accents."

Phase 2: Geolocation Tracking Middleware

"Write a Next.js middleware file that intercepts inbound page requests. Extract the user's IP Address, user-agent string (to identify browser and device type), and use request headers or a free GeoIP service to resolve the visitor's City and State. Write a server action that logs this metadata safely into a Supabase PostgreSQL table called 'site_traffic' on every interaction."

Phase 3: Stripe Manual Authorization & Order Logging

"Integrate the Stripe API into our Next.js backend for order checkout. The checkout must place an authorization hold on the customer's credit card instead of capturing funds immediately (set capture_method to manual). On a successful authorization transaction, write the order to the Supabase 'fruit_orders' table with a status of 'pending_approval' and link it to the customer profile."

Phase 4: Admin Dashboard Workflow Engine

"Build a secure Admin Dashboard page located at '/admin' protected by a simple Magic Link authentication handler. The dashboard must query and display all entries from 'fruit_orders' matching a 'pending_approval' status. Each listed order must display the customer details, purchase items, and estimated metrics alongside two action items: an 'Approve' button which triggers the Stripe payment capture function and updates status to 'approved', and a 'Cancel' button which releases the credit hold and updates status to 'cancelled'."